

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Zambrano Cedeño. Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: María Dolores Zambrano Varela

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD: # 8: Desarrollo del Backend y Base de Datos (API REST)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad _4_

Actividad # _8_

(Desarrollo de la actividad propuesta)

El estudiante deberá desarrollar el servicio web que dará vida al Help Desk. Deberá crear una base de datos para almacenar los incidentes reportados y construir una API REST que permita realizar las cuatro operaciones fundamentales (CRUD: Crear, Leer, Actualizar y Eliminar tickets).

1. Configuración del Entorno Backend

Se verificó el entorno de ejecución de **Node.js** y **NPM**, procediendo a la inicialización del proyecto backend mediante el comando `npm init -y` para la generación del archivo `package.json`. Posteriormente, se instalaron las dependencias esenciales para la arquitectura del servidor (`express`, `mongoose`, `dotenv`, `cors`, `bcryptjs`, `jsonwebtoken`).

```
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend> node -v
v24.18.0
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend> npm.cmd -v
11.16.0
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend> npm.cmd init -y
Wrote to C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
```

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Instalación de dependencia

```
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend> npm.cmd install express mongoos
e dotenv cors bcryptjs jsonwebtoken

added 103 packages, and audited 104 packages in 13s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend>
```

```
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend> npm.cmd install --save-dev node
mon

added 25 packages, and audited 129 packages in 5s

35 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend>
```



2. Configuración de la Base de Datos NoSQL

Se configuró una base de datos documental local utilizando MongoDB e interactuando mediante Mongoose. Se creó la colección tickets dentro de la base de datos helpdesk con el esquema requerido para la gestión de incidentes:

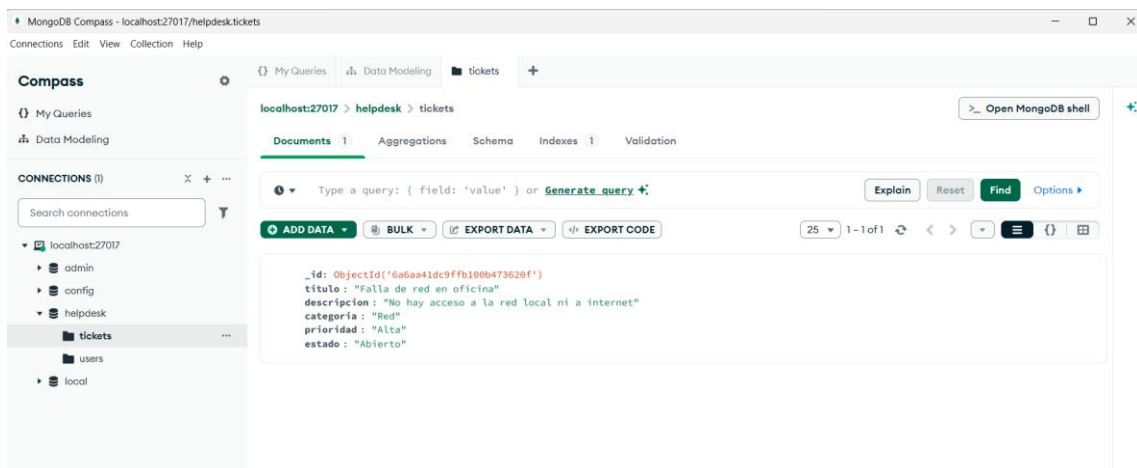
- título: String
- descripción: String
- categoria: Enum (Red, Hardware, Software)
- prioridad: Enum (Alta, Media, Baja)
- estado: Enum (Abierto, En Progreso, Cerrado)

Código del Modelo (backend/models/Ticket.js):

```
JS Ticket.js ×
backend > models > JS Ticket.js > ...
1  const mongoose = require('mongoose');
2
3  const ticketsSchema = new mongoose.Schema({
4    titulo: { type: String, required: true },
5    descripcion: { type: String, required: true },
6    categoria: { type: String, required: true },
7    prioridad: { type: String, default: 'Media' },
8    estado: { type: String, default: 'Abierto' },
9    fechaCreacion: { type: Date, default: Date.now }
10  });
11
12  module.exports = mongoose.model('Ticket', ticketsSchema);
```

Captura de la terminal ejecutando MongoDB:

```
Node.js v24.18.0
PS C:\Users\Asus\OneDrive\Documentos\Tareas de DESARROLLO DE SISTEMAS INFORMATICOS\HelpDesk\backend> node app.js
◇ injected env (0) from .env // tip: ~ auth for agents [www.vestauth.com]
Servidor HelpDesk corriendo en el puerto 5000
MongoDB Conectado: 127.0.0.1
```





3. Desarrollo de la API RESTful y Controladores

Se implementó una arquitectura de API RESTful utilizando rutas y controladores en Node.js/Express. Se definieron los endpoints necesarios para manipular los tickets de soporte en MongoDB mediante las operaciones HTTP estándar (GET, POST, PUT, DELETE).

Método HTTP	Ruta Endpoint	Descripción	Estado Esperado
POST	/api/tickets	Crear un nuevo ticket de soporte	201 Created
GET	/api/tickets	Obtener la lista completa de tickets	200 OK
GET	/api/tickets/:id	Obtener un ticket específico por su ID	200 OK / 404 Not Found
PUT	/api/tickets/:id	Actualizar los datos/estado de un ticket	200 OK
DELETE	/api/tickets/:id	Eliminar un ticket de la base de datos	200 OK

Captura POST (Crear ticket):

- Petición a: `http://localhost:5001/api/tickets` o el puerto que use.
- En la pestaña Body -> raw -> JSON, enviando un JSON

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:5000/tickets`
- Body (raw JSON):**

```
{  "titulo": "Fallo de conexion en red Wi-Fi",  "descripcion": "El departamento de ventas no tiene acceso a la red local.",  "categoria": "Red",  "prioridad": "Alta",  "estado": "Abierto"}
```
- Response:** 201 Created (149 ms, 531 B)
- Response Body (JSON):**

```
{  "titulo": "Fallo de conexion en red Wi-Fi",  "descripcion": "El departamento de ventas no tiene acceso a la red local.",  "categoria": "Red",  "prioridad": "Alta",  "estado": "Abierto",  "_id": "6a69fc69d4547282c3b34fcb",  "fechaCreacion": "2026-07-29T13:13:13.557Z",  "__v": 0}
```



Obtener todos los tickets

GET Get data

GET http://localhost:5000/tickets

Body

200 OK • 97 ms • 528 B

Request successful. The server has responded as required.

```
1 [
2   {
3     "_id": "6a69fc69d4547282c3b34fcb",
4     "titulo": "Fallo de conexion en red Wi-Fi",
5     "descripcion": "El departamento de ventas no tiene acceso a la red local.",
6     "categoria": "Red",
7     "prioridad": "Alta",
8     "estado": "Abierto",
9     "fechaCreacion": "2026-07-29T13:13:13.557Z",
10    "_v": 0
11  }
12 ]
```

Consultar un ticket por ID

GET Get data

GET http://localhost:5000/tickets/6a69fc69d4547202c3b34fcb

Body

200 OK • 51 ms • 526 B

```
1 {
2   "_id": "6a69fc69d4547282c3b34fcb",
3   "titulo": "Fallo de conexion en red Wi-Fi",
4   "descripcion": "El departamento de ventas no tiene acceso a la red local.",
5   "categoria": "Red",
6   "prioridad": "Alta",
7   "estado": "Abierto",
8   "fechaCreacion": "2026-07-29T13:13:13.557Z",
9   "_v": 0
10 }
```



Crear un nuevo ticket

The screenshot shows the Postman interface with a GET request to `http://localhost:5000/tickets/6a69fc69d4547202c3b34fcb`. The response is a 200 OK status with a JSON body containing ticket details.

```
{  "_id": "6a69fc69d4547202c3b34fcb",  "titulo": "Fallo de conexion en red Wi-Fi",  "descripcion": "El departamento de ventas no tiene acceso a la red local.",  "categoria": "Red",  "prioridad": "Alta",  "estado": "Abierto",  "fechaCreacion": "2026-07-29T13:13:13.557Z",  "__v": 0}
```

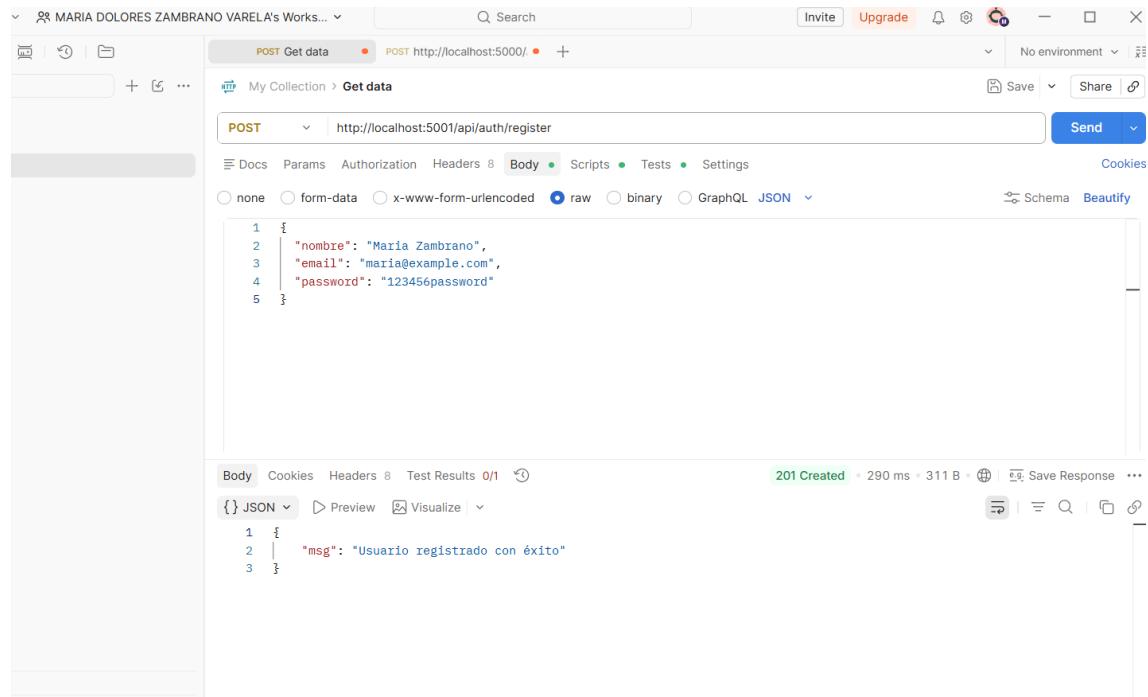
Eliminar un ticket

The screenshot shows the Postman interface with a DELETE request to `http://localhost:5000/tickets/6a69fc69d4547202c3b34fcb`. The response is a 200 OK status with a JSON body containing a success message.

```
{  "mensaje": "Ticket eliminado correctamente"}
```



Ruta del registro de usuarios



4. Control de Versiones y Metodología Gitflow

El control de código fuente del proyecto se gestionó mediante Git y se alojó en GitHub en el repositorio público oficial.

Se aplicó la metodología de ramificación **Gitflow**, organizando el trabajo en el siguiente esquema:

- **main:** Rama de producción con la versión estable del proyecto.
- **develop:** Rama de integración continua para la incorporación de nuevas funcionalidades.
- **feature/backend-api-nueva:** Rama de trabajo individual donde se construyó el servidor, modelos, controladores y rutas de la API.

Se realizó la fusión exitosa de los cambios mediante un **Pull Request / Merge** hacia la rama principal, manteniendo una historia de commits clara y excluyendo archivos innecesarios (node_modules, .env) mediante el archivo .gitignore.



GitHub repository view for **DESARROLLO-DE-SISTEMAS-INFORMATICOS-MDZV** by user **mzambrano0321**. The repository is public and has 8 branches and 0 tags. The main branch is selected.

Switch branches/tags

- main (default)
- develop
- feature/backend-api
- feature/backend-api-nueva
- feature/disenio-sistema
- feature/maquetacion-html
- feature/responsive-layout
- feature/ui-kit

Commits

- mzambrano0321/develop · 6ddc763 · 9 hours ago · 13 Commits
- Se agregan diagramas UML, secuencial, casos de usoso arquit... 2 months ago
- feat: actualizar backend completo 9 hours ago
- Se corrigen de forma definitiva las advertencias de la W3C last month

Add a README

Interested in this repository understand your project.

[Add a README](#)

GitHub repository view for **DESARROLLO-DE-SISTEMAS-INFORMATICOS-MDZV** by user **mzambrano0321**. The repository is public and has 8 branches and 0 tags. The main branch is selected.

Files

- DIAGRAMAS: Se agregan diagramas UML, secuencial, casos de usoso arquit... 2 months ago
- HelpDesk: feat: actualizar backend completo 17 minutes ago
- unidad2_maquetacion-html: Se corrigen de forma definitiva las advertencias de la W3C last month

README

Add a README

Help people interested in this repository understand your project.

[Add a README](#)